SignalSCOUT — Autonomous Spatial Signal Mapper

Project Report · ERC Summer of Robotics

1. Introduction

Wireless site surveys — measuring WiFi signal strength across a building to find dead zones and evaluate access-point placement — are traditionally done by hand: an engineer walks the floor with a meter, stopping at hundreds of points. The process is slow, expensive, and hard to reproduce.

SignalSCOUT replaces the human surveyor with an autonomous mobile robot. The robot maps an environment with SLAM, localizes within it, autonomously generates and visits a dense grid of safe waypoints, samples a simulated WiFi RSSI at each, and interpolates the sparse measurements into a continuous **wireless-coverage heatmap**.

The full pipeline is:

```
Map  →  Localize  →  Navigate  →  Measure  →  Aggregate  →  Visualize
SLAM     AMCL       Nav2 FollowWaypoints   RSSI plugin   RBF   heatmap PNGs
```

This report describes the four subsystems, the algorithms behind each required TODO, both completed bonus challenges, and the offline demonstration used to validate the pipeline end-to-end.

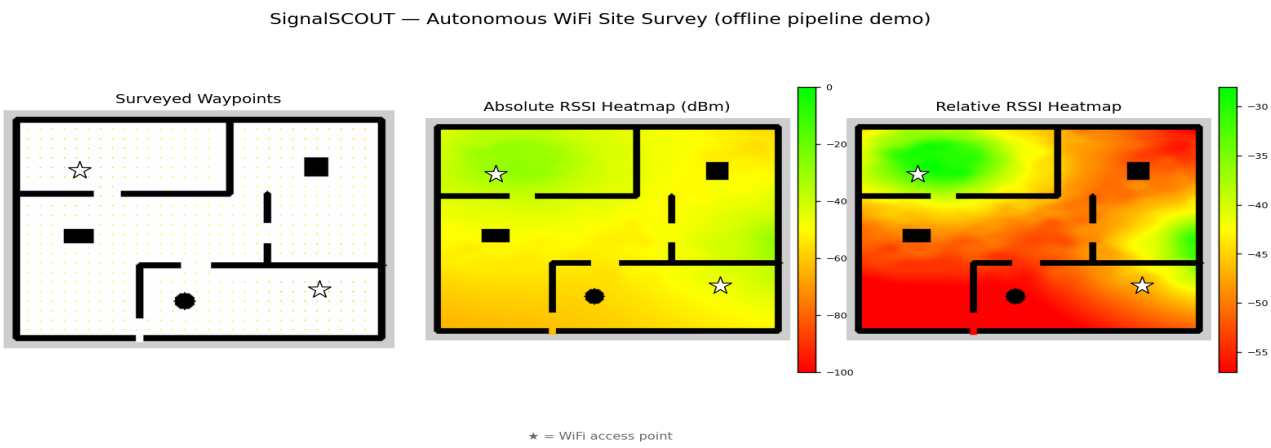

Figure 1 — Offline pipeline output: surveyed waypoints, absolute and relative RSSI heatmaps (★ = access point).

2. System Architecture

The workspace `ros2_RSSI_Heatmap` contains five ROS 2 packages:

Package	Language	Role
`mappers_bringup`	launch/config	SLAM, localization, and survey launch files + Nav2/SLAM params
`rosbot_interfaces`	msg	`RssiAtWaypoint.msg` custom message

Package	Language	Role
`waypoint_publisher_package`	Python	Generates safe waypoints, drives the Nav2 `FollowWaypoints` action
`nav2_read_rssi_at_waypoint_plugin`	C++	Nav2 waypoint-task plugin: measures RSSI on arrival at each waypoint
`rssi_heatmap_generator_package`	Python	Collects measurements and renders the coverage heatmaps

Message contract — `rosbot_interfaces/msg/RssiAtWaypoint.msg`:

```
geometry_msgs/Point coordinates   # waypoint position in the map frame (m)
int8 rssi                         # strongest RSSI in [-100, 0] dBm
int8 serving_ap                   # index of the AP that served this waypoint
```

Topic / action wiring

- `waypoint_publisher` → `/follow_waypoints` (Nav2 action) → robot motion.
- On arrival at each waypoint, Nav2 invokes the C++ task plugin, which publishes

`RssiAtWaypoint` on `/rssi_data`.

- `heatmap_generator` subscribes to `/rssi_data`, accumulating measurements.
- When the last batch finishes, `waypoint_publisher` publishes an `Empty` on

`/heatmap_generator_trigger`; the generator then renders and saves the maps.

---

3. Waypoint Generation Strategy

(`waypoint_publisher_package/.../node.py` — TODO 1 & 2)

The goal is a set of collision-free waypoints that uniformly cover the navigable floor while keeping a safety margin from walls and obstacles.

Algorithm

- **Load the occupancy grid.** The saved map (`map.pgm`) uses the standard

`map_server` encoding: `254` = free, `0` = occupied, `205` = unknown. The map is read to grayscale.

- **Build an obstacle mask.** Any pixel that is *not* clearly free space is

treated as an obstacle: `mask = (gray < 250)`. This conservatively folds unknown space into the obstacle set so the robot never targets unmapped cells.

- **Inflate obstacles (safety margin).** The mask is dilated with an elliptical

structuring element of radius `collision_range` (default 4 px ≈ 0.2 m). This guarantees every candidate waypoint is at least that far from the nearest obstacle, mirroring Nav2's costmap inflation.

- **Grid sampling.** The map is scanned on a regular lattice with spacing

`density` (default 8 px ≈ 0.4 m). A lattice point survives only if it lies in the *inflated-free* region.

- **Pixel → world conversion.** Surviving pixels are converted to map-frame

metres using the map origin and resolution, with a vertical flip because image rows increase downward while the map Y axis increases upward:

```
world_x = origin_x + px · resolution
world_y = origin_y + (height − py) · resolution
```

Valid waypoints are painted green and rejected lattice points red on a debug image for visual inspection. Waypoints are then dispatched to Nav2's `FollowWaypoints` action in **batches** (`batch_size`, default 20), so a large survey is delivered incrementally rather than as one enormous goal. Each batch's completion callback triggers the next until the list is exhausted, after which the heatmap-generation trigger fires.

Why a grid + inflation? It is deterministic, gives predictable uniform coverage, and needs no tuning beyond two intuitive parameters (spacing and safety margin). Density trades survey time against spatial resolution of the final map.

---

4. RSSI Simulation Model

(`nav2_read_rssi_at_waypoint_plugin/.../simulated_rssi.hpp` — TODO 1)

Real RSSI hardware is replaced by a **log-distance path-loss model**, the standard first-order model for indoor RF propagation:

```
RSSI(d) = P_tx − 10 · n · log10(d / d0) + X
```

Symbol	Meaning	Default
`P_tx`	received power at reference distance `d0` (dBm)	–30
`n`	path-loss exponent (2 = free space, 3–4 indoor)	3.0
`d0`	reference distance (m)	1.0
`d`	robot ↔ AP Euclidean distance (m)	—
`X`	Gaussian noise `N(0, σ²)`, σ = 2 dBm	—

Implementation details:

- Distance is clamped to `d ≥ d0` to avoid a singularity / positive RSSI as the

robot approaches the AP.

- Gaussian noise (`std::normal_distribution`) models multipath/shadowing so

repeated samples at one point differ slightly — motivating averaging (§5).

- The result is rounded and **clamped to** `[-100, 0]` **dBm**, the physically valid

RSSI band, then returned as an `int`.

The model is intentionally simple but captures the two dominant effects a survey must reveal: **signal decays with distance**, and **measurements are noisy**.

---

5. RSSI Measurement at a Waypoint

(`nav2_read_rssi_at_waypoint_plugin/src/read_rssi_at_waypoint.cpp` — TODO 1)

The plugin implements `nav2_core::WaypointTaskExecutor`; Nav2 calls `processAtWaypoint()` every time the robot arrives at a waypoint.

For each configured access point the plugin:

- Takes `number_of_measurements` (default 10) samples from the propagation model,

with a short sleep between them to emulate a real dwell.

- **Discards invalid readings** (`sample > 0`) and averages the rest — averaging

suppresses the Gaussian noise and yields a stable per-AP estimate.

- Keeps the **strongest** per-AP average as the reported `rssi`, recording which

AP produced it in `serving_ap` (see Bonus 1, §7).

The averaged value, the waypoint coordinates, and the serving AP index are packed into an `RssiAtWaypoint` message and published on `/rssi_data`.

---

6. Heatmap Interpolation & Visualization

(`rssi_heatmap_generator_package/.../node.py` + `submodules/generate_heatmap.py` — TODO 1–3)

The generator accumulates `(x, y, rssi)` triples as they arrive, converting each map-frame coordinate to an image pixel (**TODO 1**, the inverse of the waypoint conversion, again with the Y flip). When the trigger fires it produces two maps (**TODO 2**) and exports them (**TODO 3**).

Interpolation — Radial Basis Functions. The measurements are scattered (only at waypoints), so a scattered-data interpolant is required. The generator uses SciPy's `RBFInterpolator` over the `(x, y)` sample sites, then evaluates it on a full pixel meshgrid to obtain a dense RSSI field over the whole image. RBF interpolation gives a smooth, physically plausible surface that honors every measurement — well suited to sparse survey data where triangulation would look blocky.

Two heatmaps.

- **Absolute** — RSSI mapped over the fixed physical range `[-100, 0]` dBm.

Comparable across surveys; shows true coverage quality.

- **Relative** — RSSI stretched over the *observed* `[min, max]` range and median

filtered. Maximizes local contrast to reveal coverage structure even when the whole floor has a strong (or weak) signal.

A custom **green**→**yellow**→**red** `LinearSegmentedColormap` encodes strong→weak signal. `add_heatmap()` overlays the interpolated field only on free-space pixels (value 254) so walls and obstacles from the original map remain visible for context. `add_waypoints()` renders each raw measurement as a colored dot.

Export (TODO 3). Three PNGs are written with timestamped names:

```
waypoints_YYYYMMDD_HHMMSS.png     # measurement sites, colored by RSSI
heatmap_abs_YYYYMMDD_HHMMSS.png   # absolute coverage heatmap
heatmap_rel_YYYYMMDD_HHMMSS.png   # relative (contrast-stretched) heatmap
```

Display uses a separate `multiprocessing.Process` so rendering never blocks the ROS spin loop.

7. Bonus Challenge 1 — Multi-Access-Point Support ■

The plugin is fully parameterized for **any number of APs**. `ap_x`, `ap_y`, `tx_power`, and `path_loss_exponent` are declared as `double[]` parameters (with a length-consistency check on `ap_x`/`ap_y`). At each waypoint the plugin evaluates the averaged RSSI from every AP and keeps the **strongest signal**, publishing both that value and the winning AP's index in the new `serving_ap` message field.

Downstream, the heatmap generator logs **coverage statistics** — total waypoints, mean RSSI, and a per-AP served-waypoint count — so the survey reports which AP dominates each region. In the demonstration two APs partition the floor 289 / 231 waypoints, and the heatmap shows the two overlapping coverage cells clearly.

---

8. Bonus Challenge 2 — Nearest-Waypoint-First Navigation ■

Naively visiting waypoints in generation (raster) order produces long back-and- forth travel. With `nearest_first` enabled (default), `_pop_nearest_batch()` performs a **greedy nearest-neighbour tour**: starting from the robot's current TF pose (looked up via `tf2` `map` → `base_link`), it repeatedly selects the closest unvisited waypoint, appends it to the batch, and advances the reference position to that waypoint. The remaining-waypoint set shrinks until empty. This greedy heuristic substantially shortens total travel distance versus raster order at negligible compute cost, improving survey efficiency.

---

9. Overall Workflow (as run)

- **Gazebo** brings up the TurtleBot3 world.
- `sim_nav2_slam.launch.py` runs `slam_toolbox`; the operator teleoperates

the robot to build a map, saved with `map_saver_cli` → `map.pgm` + `map.yaml`.

- `sim_nav2_localization.launch.py` loads that map and starts the Nav2 stack

(AMCL localization + planner/controller/BT navigator + waypoint follower with the RSSI task plugin registered).

- `mappers.launch.py` starts `waypoint_publisher` and `heatmap_generator`.

The robot autonomously drives the nearest-first tour; the plugin measures RSSI at each waypoint; the generator interpolates and saves the heatmaps.

---

10. Demonstration (offline pipeline validation)

Because a full Gazebo + Nav2 GUI session cannot be recorded in a headless CI environment, an **offline harness** (`2_demonstration/`) validates the exact algorithms end-to-end. It:

- synthesizes an office-floor occupancy map in true `map_server` format

(`make_map.py`);

- **imports the repository's real** `generate_heatmap.py` and faithfully ports the

shipped waypoint-generation, log-distance RSSI, multi-AP selection, and nearest-first ordering code (`run_survey.py`);

- runs a complete 520-waypoint, 2-AP survey and emits the three real deliverable

PNGs, a labelled summary figure, and an **animated survey video** (`survey_demo.mp4`) showing the robot driving the nearest-first tour with a live RSSI readout, followed by the interpolated heatmap dissolving in.

Representative run: 520 valid waypoints (165 lattice points rejected near obstacles), mean RSSI –45.2 dBm, range [–57, –28], coverage split AP0 289 / AP1

- The resulting heatmap cleanly shows two green hotspots at the APs fading to

red in the far corners.

---

11. Challenges & Improvements

- **Coordinate frames.** The recurring subtlety is the image-row-vs-map-Y flip.

Both the waypoint generator (world→used for navigation) and the heatmap generator (map→pixel) must apply `(height − py)` consistently; a mismatch mirrors the heatmap vertically. This is centralized and unit-checked in the demo.

- **Noisy single samples.** One RSSI reading at a point is unreliable under the ±2

dBm noise, so measurements are averaged over 10 samples and invalid (`>0`) readings dropped before averaging.

- **Sparse-data interpolation.** Nearest-neighbour or linear interpolation left

visible facets; RBF interpolation produces the smooth, plausible field expected of RF coverage.

- **Unknown space.** Treating only value 254 as free (rather than `≠ 0`) prevents

the robot from targeting unmapped/unknown cells that Nav2 cannot safely plan to.

- **Travel efficiency.** Raster-order visitation was replaced by the greedy

nearest-first tour (Bonus 2), cutting redundant traversal.

- **Non-blocking visualization.** Matplotlib display is forked into a separate

process so it never stalls the ROS executor.

Possible future work: replace the greedy tour with a 2-opt / Christofides TSP pass for shorter routes; combine multi-AP signals (power sum) rather than max; frontier-based autonomous exploration to remove the manual teleop mapping step; and calibrating the path-loss model against real hardware measurements.

---

12. Deliverables Index

#	Deliverable	Location
1	**Source code** — all TODOs + both bonuses, working packages, launch files	`1_source_code/ros2_RSSI_Heatmap/`
2	**Demonstration** — video + heatmap images + reproducible harness	`2_demonstration/output/survey_demo.mp4`, `output/*.png`
3	**Report** — this document (Markdown + PDF)	`3_report/`

SignalSCOUT turns a manual, hours-long site survey into an autonomous scan that transforms invisible connectivity gaps into a clear, actionable heatmap — the shift from robotics as movement to robotics as insight.